

VayaVia Demand Engine: Lovable Prompts, Built-In Database & Antigravity Deployment

Overview

This document contains the **exact, copy-paste-ready Lovable prompts** to build the VayaVia Demand Engine admin panel — a complete React + Tailwind CSS application with a **built-in SQLite/IndexedDB database** (no Supabase, no external DB). All 37 data tables from the 5 Excel sheets are replicated as local database tables inside the app itself. After building in Lovable, the code downloads to GitHub, then gets further developed in Google Antigravity IDE connected to the VPS via SSH. The final system runs on Vercel (frontend) connected to the VPS (backend engine) via n8n webhook APIs.

Architecture: No Supabase — Local DB + VPS PostgreSQL

Since Supabase is off the table, the architecture splits into two databases:

1. Admin Panel (Vercel) → Built-in IndexedDB via Dexie.js

- All 37 seed data tables (agents, competitors, influencers, etc.) stored locally in browser
- Works offline, zero cost, no external dependency
- Dashboard metrics, filters, search — all local-first

2. VPS Backend → PostgreSQL (self-hosted in Docker)

- Live demand signals, AI-scored leads, campaign logs, forecasts
- n8n writes to this database continuously
- Admin panel reads from VPS PostgreSQL via n8n webhook API endpoints

3. Bridge: n8n Webhooks

- Admin panel calls `https://n8n.vayavia.com/webhook/*` endpoints to fetch live data
- n8n responds with JSON from PostgreSQL
- No Supabase needed — n8n IS the API layer[1][2]

Lovable Prompt Sequence — Complete Build

PROMPT 1: Project Initialization + Theme + Layout

Copy-paste this into Lovable as the first prompt:

Create a React + TypeScript + Tailwind CSS admin dashboard called "VayaVia

THEME:

- Dark background: #0f0f13
- Card backgrounds: #1a1a24
- Primary accent: #722F37 (wine red)
- Secondary accent: #9B4DCA (purple)
- Success: #10B981 (green)
- Warning: #F59E0B (amber)
- Danger: #EF4444 (red)
- Text primary: #F1F1F1
- Text secondary: #9CA3AF
- Font: Inter (Google Fonts)

LAYOUT:

- Fixed left sidebar (240px wide, collapsible to 64px)
- Sidebar navigation items with icons:
 1. Dashboard (home icon) — default active
 2. Signals (radar icon)

3. Leads (users icon)
 4. Campaigns (megaphone icon)
 5. Forecasts (chart-line icon)
 6. Content Calendar (calendar icon)
 7. Competitors (swords icon)
 8. Data Hub (database icon)
 9. System Logs (terminal icon)
 10. Settings (gear icon)
- Top bar: "VayaVia Demand Engine" title left, system status indicator (green circle)
 - Main content area with padding

ROUTING:

- Use React Router for all pages
- Each sidebar item routes to its own page

Add a small VayaVia logo text in the sidebar header using the wine-red color

PROMPT 2: Local Database with Dexie.js + All 37 Seed Tables

Add a local database using Dexie.js (IndexedDB wrapper) with the following

DATABASE NAME: "VayaViaDemandEngine"

SEED DATA TABLES (pre-loaded with data):

1. travel_agents — columns: sr_no, company_name, contact_person, phone, email, website, address, rating, reviews, status, created_at, updated_at
(This will hold 330 rows — leave empty for now, will import via CSV)
2. nashik_tourism_db — columns: business_name, category, address, phone, email, website, rating, reviews, status, created_at, updated_at
(31 rows seed data)
3. coworking_startups — columns: name, type, address, phone, email, website, rating, reviews, status, created_at, updated_at
(22 rows)
4. travel_influencers — columns: name, platform, followers, engagement_rate, niche, location, status, created_at, updated_at
(22 rows)

5. competitors — columns: company_name, website, founded, funding, prop
(16 rows)

6. listing_platforms — columns: platform, website, type, monthly_traffic, listi
(21 rows)

7. remote_startups — columns: company_name, website, industry, city, fundi
(17 rows)

8. spiritual_sites — columns: temple_site, city_state, annual_footfall, peak_m
(17 rows)

9. events_2026 — columns: event_name, date_2026, location, expected_footfa
(14 rows)

10. nomad_communities — columns: community_name, url, type, members,
(22 rows)

11. property_listings — columns: name, type, phone, email, property_type, lo
(22 rows)

12. travel_pr_journalists — columns: name, publication, beat_focus, email, tr
(22 rows)

13. wedding_event_planners — columns: company_name, type, city, phone, c
(22 rows)

14. nashik_content_creators — columns: creator_name, platform, followers,
(22 rows)

15. maharashtra_tourism_govt — columns: department, designation, name, c
(17 rows)

16. family_mom_influencers — columns: name, platform, followers, niche, i
(22 rows)

17. photo_video_drone — columns: name, specialization, instagram, phone,

(22 rows)

18. bus_operators — columns: operator_name, type, route, departure_time, a
(17 rows)

19. wellness_yoga_retreats — columns: center_name, type, location, phone, e
(22 rows)

20. real_estate_developers — columns: developer_name, type, location, phon
(17 rows)

JOURNEY MAPPING TABLES:

21. journey_stage1_dream — columns: platform_channel, url_handle, conter
(14 rows)

22. journey_stage2_research — columns: search_query, search_volume, user
(12 rows)

23. journey_stage3_shortlist — columns: destination, nomadlist_score, foreig
(10 rows)

24. journey_stage4_compare — columns: platform, market_share_india, fore
(16 rows)

25. journey_stage5_book — columns: concern_area, frequency, foreign_touri
(18 rows)

26. journey_stage6_experience — columns: touchpoint, foreign_expectation,
(17 rows)

27. journey_stage7_advocate — columns: channel, usage_rate, content_type, ,
(23 rows)

28. traveler_segments — columns: segment_name, stay_duration, budget_rar
(8 rows)

29. country_analysis — columns: country, tourist_volume, cultural_traits, bo

(7 rows)

MARKETING TABLES:

30. content_strategy_50 — columns: piece_number, title, format, platform, job_id
(51 rows)

31. social_media_calendar — columns: day, date, platform, content_type, post_id
(60 rows)

32. email_marketing — columns: email_number, subject_line, preview_text, job_id
(62 rows)

DEMAND INTELLIGENCE TABLES:

33. flight_data_tools — columns: tool_name, data_provided, access_method, job_id
(85 rows)

34. life_event_triggers — columns: event_type, avg_time_to_trip, booking_preferences
(99 rows)

35. weather_data_sources — columns: source, data_available, pricing, api_access
(39 rows)

36. cost_analysis — columns: category, free_tier, starter_cost, professional_cost
(23 rows)

37. implementation_roadmap — columns: week, category, action_items, tools
(38 rows)

LIVE ENGINE TABLES (empty — filled by VPS):

38. demand_signals — columns: signal_type, source, raw_data, ai_classification

39. leads — columns: source, name, email, phone, city, life_event, ai_score, address

40. visitor_profiles — columns: session_id, pages_viewed, time_on_site, ai_segment

41. competitor_prices — columns: platform, property_name, price_per_night

42. weekly_forecasts — columns: forecast_period_start, forecast_period_end, demand

43. workflow_logs — columns: workflow_name, status, duration_ms, started_at

- 44. campaign_logs — columns: campaign_name, trigger_condition, trigger_value
- 45. decision_logs — columns: decision_type, input_data, ai_output, action_taken

Create a database service file (db.ts) that initializes all tables and exports the

PROMPT 3: Dashboard Home Page with Live Metrics

Build the Dashboard (home) page with the following layout:

ROW 1 — 6 metric cards in a grid (3 columns, 2 rows):

- "Signals Today" — large number, radar icon, wine-red accent, shows count of active signals
- "Hot Leads" — large number, fire icon, orange accent, count from leads with high demand
- "Active Campaigns" — large number, megaphone icon, purple accent, count of ongoing campaigns
- "Forecast Score" — large text (Low/Medium/High/Very High), chart icon, color-coded background
- "Website Visitors" — large number, eye icon, blue accent, placeholder "—" if data is missing
- "AI Model Status" — text "Online" or "Offline", cpu icon, green or red, placeholder "—" if unknown

ROW 2 — Two panels side by side:

LEFT (60% width): "Recent Activity" feed — chronological list of latest 20 events

RIGHT (40% width): "Quick Actions" panel with buttons:

- "Force Weather Check" (cloud icon)
- "Run Trend Scan" (trending icon)
- "Rescore All Leads" (refresh icon)
- "Test AI Model" (robot icon)
- "Sync Live Data" (download icon)

Each button calls a configurable webhook URL (stored in Settings) via fetch API

ROW 3 — Full width:

"Demand Signal Timeline" — horizontal scrollable timeline showing the last 24 hours

All metric cards should have a subtle pulse animation when data changes. A

PROMPT 4: Signals Page

Build the Signals page:

TOP: Filter bar with:

- Signal Type dropdown: All, Weather, AQI, Trend, Event, Competitor, Life Ev
- Urgency dropdown: All, Low, Medium, High, Critical
- Date range picker (last 24h, 7d, 30d, custom)
- Search box (searches across all signal fields)
- "Export CSV" button

MAIN: Data table showing demand_signals with columns:

- Timestamp (formatted as "Mar 13, 2:15 AM")
- Signal Type (icon + text: cloud for weather, wind for AQI, trending for trend)
- Source (API name)
- Urgency (color-coded badge: green=low, yellow=medium, red=high, black w
- Target Segment (badge with segment name)
- Campaign Triggered (green checkmark or gray X)
- Details (expandable row showing raw_data and ai_classification as formatt

Table features: sortable columns, pagination (25 per page), row click expand

RIGHT SIDEBAR (collapsible): "Signal Intelligence" panel showing:

- Signals by type (mini pie chart using recharts)
- Signals by urgency (horizontal bar chart)
- Avg signals per day this week
- Most active signal type

PROMPT 5: Leads Page

Build the Leads page:

TOP: Filter bar with:

- Source dropdown: All, Clay, LinkedIn, Website, Agent Referral, Manual
- Segment dropdown: All, Digital Nomad, Luxury Couple, Spiritual Seeker, Co
- Score range slider (0-100)
- Status dropdown: All, New, Contacted, Nurturing, Booked, Lost

- Search box
- "Export CSV" button
- "Add Lead" button (opens modal form)

MAIN: Data table for leads:

- Name (bold)
- Source (badge)
- City
- Life Event (italic, truncated to 40 chars)
- AI Score (number with color gradient background: 0-40 red, 41-60 yellow, 61-80 green, 81-100 blue)
- Segment (colored badge matching segment)
- Status (dropdown that updates the record directly)
- Created (time ago)

ROW CLICK: Opens a detail drawer from the right (400px wide) showing:

- Full lead info
- AI-generated message in a copyable text block with "Copy" and "Send WhatsApp" buttons
- AI reasoning text
- Timeline of status changes
- "Override Score" input + save button
- "Mark as Booked" / "Mark as Lost" action buttons

BOTTOM: Lead score distribution chart (histogram) and leads by segment (donut chart)

PROMPT 6: Campaigns Page — Pre-Planned Scenario Triggers

Build the Campaigns page showing pre-planned scenario triggers:

Layout: Grid of campaign cards (2 columns), each card represents a scenario trigger.

CARD 1 — "Pollution Escape"

- Trigger: AQI > 200 in any feeder city (Delhi, Mumbai, Pune, Bangalore)
- ON/OFF toggle (large, prominent)
- Status indicator: Armed (green), Triggered (orange pulse), Cooldown (gray)
- Last triggered: date or "Never"
- Cooldown: 72 hours

- Template preview: Email + WhatsApp message preview
- "Edit Template" button
- "Force Trigger" button (for testing)

CARD 2 — "Clear Weekend Push"

- Trigger: Nashik Sat-Sun forecast clear + temp 22-30°C
- Same structure as above
- Cooldown: 7 days

CARD 3 — "Trend Spike Alert"

- Trigger: Any keyword up >30% week-over-week
- Cooldown: 48 hours

CARD 4 — "Competitor Sellout"

- Trigger: >80% Nashik properties unavailable on Booking.com
- Cooldown: 5 days

CARD 5 — "Festival Season"

- Trigger: Calendar match (show upcoming festivals from events_2026 table)
- Fires 14 days before event
- Cooldown: Per event

CARD 6 — "Life Event Outreach"

- Trigger: Clay detects promotion/birthday/anniversary AND lead score > 80
- Cooldown: Per lead (once)

CARD 7 — "Flight Spike"

- Trigger: Pune/Mumbai flight searches up >25%
- Cooldown: 5 days

Each card stores its config in a local "campaigns" table with fields: name, trig

BOTTOM: Campaign execution log — table showing campaign_logs (timestan

PROMPT 7: Data Hub — Browse All 37 Seed Tables

Build the Data Hub page:

This is a data browser for all 37 seed data tables stored in the local database.

LEFT PANEL (250px): Tree-style navigation grouped by category:

▣ LEADS & AGENTS

- Travel Agents (330)
- Wedding Event Planners (22)
- Maharashtra Tourism Govt (17)

▣ NASHIK ECOSYSTEM

- Nashik Tourism Database (31)
- Property Listings (22)
- Coworking Startups (22)
- Photo Video Drone (22)
- Real Estate Developers (17)

▣ INFLUENCERS & MEDIA

- Travel Influencers (22)
- Nashik Content Creators (22)
- Family Mom Influencers (22)
- Travel PR Journalists (22)

▣ COMPETITIVE INTEL

- Competitors Deep Dive (16)
- Listing Platforms (21)
- Remote Startups India (17)

▣ TOURISM ASSETS

- Spiritual Sites (17)
- Events 2026 (14)
- Digital Nomad Communities (22)
- Wellness Yoga Retreats (22)
- Bus Operators Routes (17)

✈ TRAVELER JOURNEY

- Stage 1 Dream (14)
- Stage 2 Research (12)
- Stage 3 Shortlist (10)
- Stage 4 Compare (16)
- Stage 5 Book (18)
- Stage 6 Experience (17)
- Stage 7 Advocate (23)
- Traveler Segments (8)
- Country Analysis (7)

□ MARKETING

- Content Strategy 50-Piece (51)
- Social Media Calendar (60)
- Email Marketing (62)

□ DEMAND INTELLIGENCE

- Flight Data Tools (85)
- Life Event Triggers (99)
- Weather Data Sources (39)
- Cost Analysis (23)
- Implementation Roadmap (38)

RIGHT PANEL: When a table is selected:

- Table name + row count as header
- Search box to filter rows
- Full data table with all columns, sortable
- Row click opens detail modal
- "Export CSV" button
- "Import CSV" button (bulk import from file — this is how the 330 travel age

The Import CSV function should:

1. Accept a .csv file
2. Auto-map CSV columns to table columns
3. Show preview of first 5 rows

4. Confirm import
5. Bulk insert into the Dexie.js table

PROMPT 8: Forecasts Page

Build the Forecasts page:

TOP: Current forecast card (large, full width):

- "14-Day Demand Outlook" header
- Large predicted_demand text (Low/Medium/High/Very High) with color
- Period: "March 14 - March 28, 2026"
- Demand drivers as tag badges
- Recommended actions as a checklist (checkboxes)

MIDDLE: "Demand History" line chart (recharts):

- X-axis: weeks
- Y-axis: demand score (1-4 mapped to Low-Very High)
- Color-coded line sections
- Data from weekly_forecasts table

BOTTOM: Two panels side by side:

LEFT: "Signal Correlation Matrix" — which signal types preceded high-demand

RIGHT: "Upcoming Events Impact" — events from events_2026 table sorted by

PROMPT 9: Content Calendar Page

Build the Content Calendar page using data from social_media_calendar and

TOP: Month/week view toggle + platform filter (All, Instagram, LinkedIn, Tw

CALENDAR VIEW:

- Monthly calendar grid
- Each day cell shows content items as colored pills:
 - Instagram = pink
 - LinkedIn = blue
 - Twitter = cyan

- Email = purple
- Facebook = navy
- Clicking a pill opens a detail modal showing: platform, content_type, full po

LIST VIEW (toggle):

- Table of all content items sorted by date
- Columns: Date, Platform, Type, Caption (truncated), Time, Status (Draft/Sche
- Status is editable via dropdown

BOTTOM: "Email Sequence" timeline — shows email_marketing items as a ho

PROMPT 10: Competitors Page

Build the Competitors page using data from the competitors table:

TOP: Competitor cards in a grid (3 columns):

Each card shows: company_name, price_range, properties count, target_audi

MIDDLE: "Live Price Monitor" table showing competitor_prices (from VPS liv

- Columns: Platform, Property, Price/Night, Availability, Review Score, Last Cl
- Color-code: green if VayaVia is cheaper, red if competitor is cheaper

BOTTOM: Comparison table — competitors side by side with VayaVia:

| Metric | VayaVia | StayVista | Zostel | SaffronStays | ... |

Auto-populated from competitors table data.

PROMPT 11: System Logs + VPS Connection Settings

Build the System Logs page:

TOP: Status cards in a row:

- n8n Status (green/red dot + "Connected"/"Disconnected")
- Ollama Model (model name + "Loaded"/"Offline")
- PostgreSQL (status + row counts)
- API Quota (calls used today / limit for each API)

MAIN: Workflow execution log table from workflow_logs:

- Columns: Workflow Name, Status (green badge=success, red badge=error, yellow badge=pending)
- Auto-refreshes every 30 seconds by calling the VPS health webhook
- Filter by status, workflow name, date range

BOTTOM: Raw log viewer — scrollable monospace text area showing latest 100 logs

Also build the Settings page with:

SECTION 1: "VPS Connection"

- n8n Base URL input (default: https://n8n.vayavia.com)
- Webhook Auth Token input (password field)
- "Test Connection" button that calls /webhook/system-health and shows result
- Status indicator

SECTION 2: "API Keys" (stored in localStorage)

- OpenWeatherMap API Key
- IQAir API Key
- SerpApi API Key
- Google Analytics Property ID
- Clay API Key
- Each has a text input + "Verify" button

SECTION 3: "Webhook Endpoints" (auto-configured from VPS URL)

- List of all webhook URLs:
 - /webhook/trigger-campaign
 - /webhook/rescore-lead
 - /webhook/force-weather-check
 - /webhook/test-ollama
 - /webhook/system-health
 - /webhook/get-signals (GET — returns recent signals)
 - /webhook/get-leads (GET — returns recent leads)
 - /webhook/get-forecasts (GET — returns forecasts)
 - /webhook/get-logs (GET — returns workflow logs)
- Each shows last response status + timestamp

SECTION 4: "Data Sync"

- "Sync Signals from VPS" button — calls /webhook/get-signals, inserts into local database

- "Sync Leads from VPS" button — same for leads
- "Full Sync" button — syncs all live tables
- Auto-sync toggle: ON/OFF + interval (30s, 1m, 5m, 15m)
- Last sync timestamp

PROMPT 12: CSV Import System + Data Seeding

Add a comprehensive CSV import system accessible from:

1. The Data Hub page ("Import CSV" button on each table)
2. Settings page under "Data Management" section

IMPORT FLOW:

Step 1: Select target table from dropdown (all 37 tables listed)

Step 2: Upload CSV file (drag-and-drop area)

Step 3: Column mapping UI — show CSV columns on left, database columns

Step 4: Preview — show first 10 rows as they will be imported

Step 5: Import — progress bar, row count, success/error count

Step 6: Done — show "X rows imported successfully" with link to view table

Also add:

- "Export All Data" button in Settings — exports all tables as a single JSON file
- "Import All Data" button — imports from JSON backup
- "Clear Table" button on each table in Data Hub (with confirmation dialog)
- "Reset to Empty" button in Settings (clears all tables, with double confirmat

This CSV import is the primary way data gets into the system. The 330 travel

Post-Lovable: Antigravity Workflow

Step 1: Download from Lovable → GitHub

After all 12 prompts are built in Lovable:[3]

1. In Lovable → Click GitHub icon → Create repo: vayavia-demand-engine
2. All code syncs to GitHub automatically

3. Clone locally: `git clone https://github.com/[your-username]/vayavia-demand-engine.git`

Step 2: Open in Antigravity IDE

Google Antigravity opens the cloned repository and provides AI-assisted development:[4][5]

```
# In Antigravity IDE:
# Open the cloned project folder

# Key files Lovable created:
src/
├── components/
│   ├── layout/
│   │   ├── Sidebar.tsx
│   │   └── TopBar.tsx
│   ├── dashboard/
│   │   ├── MetricCard.tsx
│   │   ├── ActivityFeed.tsx
│   │   └── SignalTimeline.tsx
│   ├── signals/
│   │   └── SignalsTable.tsx
│   ├── leads/
│   │   ├── LeadsTable.tsx
│   │   └── LeadDrawer.tsx
│   ├── campaigns/
│   │   └── CampaignCard.tsx
│   └── datahub/
│       ├── TableBrowser.tsx
│       └── CsvImporter.tsx
├── db/
│   └── database.ts      ← Dexie.js database with all 45 tables
├── hooks/
│   └── useVpsSync.ts    ← Hook for VPS webhook communication
├── pages/
│   ├── Dashboard.tsx
│   └── Signals.tsx
```

```

|   |— Leads.tsx
|   |— Campaigns.tsx
|   |— Forecasts.tsx
|   |— ContentCalendar.tsx
|   |— Competitors.tsx
|   |— DataHub.tsx
|   |— SystemLogs.tsx
|   |— Settings.tsx
|   └─ lib/
|       |— api.ts           ← Webhook API calls to VPS
|       └─ csvImport.ts     ← CSV parsing and import logic

```

Step 3: Antigravity Prompts for Further Development

Open Antigravity chat and give these prompts to enhance what Lovable built:

Antigravity Prompt 1 — VPS Sync Engine:

In `src/hooks/useVpsSync.ts`, build a sync engine that:

1. Reads the VPS base URL from `localStorage` (set in Settings page)
 2. Every X seconds (configurable), calls these n8n webhook endpoints:
 - `GET /webhook/get-signals?since={lastSyncTimestamp}`
 - `GET /webhook/get-leads?since={lastSyncTimestamp}`
 - `GET /webhook/get-logs?since={lastSyncTimestamp}`
 3. Inserts new records into the local `Dexie.js` tables
 4. Updates the "Last Synced" timestamp in the top bar
 5. Shows a toast notification for any new high-urgency signal or hot lead
 6. Handles errors gracefully — shows "VPS Disconnected" in top bar if webh
- Make it a React context provider so any component can access sync state.

Antigravity Prompt 2 — Real-Time Notification System:

Add a notification system:

1. Bell icon in the top bar with unread count badge
2. Notification dropdown showing recent events:
 - 🚨 Critical signal detected: "Delhi AQI 340"

- ❌ Hot lead scored 85: "Rahul Mehta — VP promotion"
- ✔️ Campaign triggered: "Pollution Escape"
- ❌ Workflow failed: "Weather Monitor — timeout"

3. Notifications come from the sync engine — any new demand_signal with 1
4. Browser push notifications for critical items (request permission on first l
5. Notification settings in Settings page — toggle which types send push notif

Antigravity Prompt 3 — Campaign Template Editor:

In the Campaigns page, when "Edit Template" is clicked on a campaign card,

1. Tabs: "Email Template" and "WhatsApp Template"
2. Rich text editor for email (use TipTap or similar)
3. Plain text editor for WhatsApp with variable placeholders: {name}, {city},
4. Preview panel showing the template with sample data filled in
5. Save to the campaigns Dexie.js table
6. "Send Test" button that calls /webhook/send-test-campaign with the template

Step 4: Connect Antigravity to VPS via SSH

To work on VPS files directly from Antigravity:[6][7]

```
# In Antigravity IDE:
# Settings → Remote Connections → Add SSH Connection
# Host: your-vps-ip
# Port: 22
# User: root
# Auth: SSH Key (paste your private key)

# Once connected, Antigravity can:
# - Browse VPS files
# - Edit n8n workflow JSONs
# - Read Docker logs
# - Run terminal commands
# - Modify Ollama Modelfiles
```

Antigravity VPS Prompts:

- > SSH into the VPS and check if all Docker containers are running. Show me
- > Open /opt/vayavia-ai/docker-compose.yml and add a health check for the O
- > Read the last 50 lines of the n8n container logs and tell me if there are any
- > Create a new n8n webhook workflow that responds to GET /webhook/get-s with the last 100 demand_signals from PostgreSQL as JSON.
- > Update the Ollama Modelfile for vayavia-signal to add a new segment called "honeymoon_couple" with scoring rules.

Step 5: Deploy to Vercel

```
# In terminal (local or Antigravity):  
cd vayavia-demand-engine  
git add .  
git commit -m "Complete demand engine admin panel"  
git push origin main  
  
# In Vercel:  
# 1. Import from GitHub → select vayavia-demand-engine repo  
# 2. Framework: Vite  
# 3. Build command: npm run build  
# 4. Output directory: dist  
# 5. No environment variables needed (all stored in browser localStorage)  
# 6. Deploy → live at admin.vayavia.com
```

Every subsequent git push auto-deploys to Vercel.[8][9]

CSV Export from Excel Sheets

To get data from the 5 Excel sheets into the Lovable-built admin panel, export each sheet tab as CSV and use the built-in CSV importer:
[10][11][12][13][14]

Sheet Tab	Target Table	Rows
Travel Agents Database	travel_agents	330
1_Nashik_Tourism_Database	nashik_tourism_db	31
2_Nashik_Coworking_Startups	coworking_startups	22
3_Indian_Travel_Influencers	travel_influencers	22
4_Competitor_Deep_Dive	competitors	16
10_Travel_Listing_Platforms	listing_platforms	21
5_Remote_Startups_India	remote_startups	17
7_Spiritual_Tourism_Sites	spiritual_sites	17
8_Events_2026_Nashik	events_2026	14
9_Digital_Nomad_Communities	nomad_communities	22
6_Nashik_Property_Listings	property_listings	22
11_Travel_PR_Journalists	travel_pr_journalists	22
13_Wedding_Event_Planners	wedding_event_planners	22
14_Nashik_Content_Creators	nashik_content_creators	22
15_Maharashtra_Tourism_Govt	maharashtra_tourism_govt	17

Sheet Tab	Target Table	Rows
16_Family_Mom_Influencers	family_mom_influencers	22
17_Nashik_Photo_Video_Drone	photo_video_drone	22
18_Bus_Operators_Routes	bus_operators	17
19_Wellness_Yoga_Retreats	wellness_yoga_retreats	22
20_Real_Estate_Developers	real_estate_developers	17
Stage 1-7 (all journey tabs)	journey_stage1 through journey_stage7	130 total
Traveler Segments Deep-Dive	traveler_segments	8
Country-of-Origin Analysis	country_analysis	7
50-Piece Content Strategy	content_strategy_50	51
Social Media Calendar	social_media_calendar	60
Email Marketing Toolkit	email_marketing	62
Flight Data tools	flight_data_tools	85
Life Event Triggers	life_event_triggers	99
Weather Data Sources	weather_data_sources	39
Cost Analysis	cost_analysis	23
Implementation Roadmap	implementation_roadmap	38

Total: 37 tables, ~1,340 rows of seed data — all imported via the CSV Import tool on the Data Hub page.

References

1. [Creating a Webhook API Endpoint with n8n - DEV Community](#) - ## 1. Setting Up a Basic Webhook Node The Webhook node serves as the entry point for external HTTP r...
2. [n8n Webhook Tutorial: Trigger Workflows from Anywhere \(2026\)](#) - Learn how to set up n8n webhooks for real-time workflow automation. Step-by-step tutorial covering a...
3. [Lovable vs Vercel: Platform Comparison Guide](#) - Cost structure: Lovable bundles AI building and hosting, while Vercel separates hosting from applica...
4. [A first look at Google's new Antigravity IDE](#) - Google's agentic AI development environment includes multiple agent models and browser automation wi...
5. [How to Set Up and Use Google Antigravity](#) - Google Antigravity is Google's free AI-powered IDE that lets developers build software using autonom...
6. [Remote SSH + Antigravity / Google Code Assist?](#) - Hello, I usually work with Remote SSH over my VPS, and so when installing Gemini Code Assist on VSC,...
7. [Beyond VS Code SSH — Remote Development in Antigravity](#) - The Workflow: Install the OpenCode CLI on your remote VPS. Run opencode serve. On your local machine...
8. [How to Deploy a Lovable Project on Vercel \(With Custom ... - YouTube](#) - In this video, I'll show you how to deploy a Lovable project on Vercel step by step. This tutorial c...
9. [How to deploy to Vercel from Loveable](#) - How to Deploy Your Lovable Project to Vercel (Step-by-Step Guide)

Are you looking to move your AI-g...

10. [6000_Indian_Travel_Agent_Business_Leads-Genspark_AI_Sheets-20260211_1858.xlsx](#) - Sheet Travel Agents Database Sr. No.,Company Name,Contact Person,Phone,Mobile,Email,Address,City,Sta...
11. [Nashik_Tourism-Startups-_and_Influencer_Data_Shee-Genspark_AI_Sheets-20260211_1855-4.xlsx](#) - Sheet

- 1NashikTourismDatabase Business
Name,Category,Address,Phone,Email,Website,Google
Rating,Instag...
12. [Inbound_Tourism_Journey-_Mapping_International_Tra-Genspark_AI_Sheets-20260211_1821-3.xlsx](#) - Sheet Executive Summary
FOREIGN TOURIST DECISION JOURNEY TO INDIA -
COMPREHENSIVE RESEARCH REPORT,Un...
13. [30-Day_Social_Calendar_and_Email_Marketing_Toolkit-Genspark_AI_Sheets-20260211_1829-2.xlsx](#) - Sheet Social Media Calendar
Day,Date,Platform,Content Type,Post
CopyCaption,Hashtags,Visual Descript...
14. [VayaVia_Nashik_Hospitality_Demand_Generation_Intel-Genspark_AI_Sheets-20260211_1850-5.xlsx](#) - Sheet Overview Index
VayaVia Data Intelligence Strategy - 12 Unconventional Data
Categories,Unnamed ...